NANYANG THE INNOVATIVE POLYTECHNIC

2026S1

IT3103 Assignment 2

1 NYP Policies

1.1 Late submission and penalty

For late submissions of 5 calendar days or less,

- If you passed the assignment/project deliverable, your score will be capped at 50% of the base score of the assignment/project deliverable.
- If you fail the assignment/project deliverable, you will be awarded a failed score.

For late submission or more than 5 calendar days,

- You will be awarded zero marks for the assignment/project deliverable.

Please note:

- It is your responsibility to ensure that your submission is complete and correct. You may wish to download a copy after submission to verify that all files are included and accurately submitted.
- If you have multiple submissions, the most recent submission will be considered as your final version and will be subjected to penalty if it is submitted late.

1.2 Academic Integrity

The following are considered acts of academic dishonesty:

- Attempting to get or provide unauthorized assistance in an assessment
- Falsifying data, information or citations
- Asking another person to complete the work that the candidate is supposed to do
- Taking and using the whole or any part of ideas, words or works of other people and passing it off as one's own work without acknowledgement of the original source

If a learner is caught cheating for the:

- First time in any assessment, he/she will fail the entire learning unit.
- Second time in any assessment, he/she will fail all learning units in that semester.
- Third time in any assessment, he/she will be removed from the Polytechnic.

2 Assignment - Building a Multimodal Agent

You've learned that a modern AI agent is built from five ingredients:

Agent = LLM + Loop + Tools + Memory + Self-Correction

In this assignment you will build a working visual agent from scratch. Your agent is powered by an open-source, locally run vision-language model, Qwen3-VL-8B, which can see images and call tools. You are tasked to:

- Create the task dataset
- Create a set of tools that are suited for the tasks
- Implement the ReAct loop (Thought - Action - Observation)
- Implement Reflexion so the agent can learn from its own mistakes and retry
- Compare the performance between plain LLM, ReAct and Reflexion across all tasks

The task is designed so that a single model call cannot solve it: the agent receives a task, decides an action, uses a tool, observes the result, and continues until the task is solved.

Model note. The reference model is Qwen3-VL-8B (4-bit/AWQ), which runs on a free Google Colab T4 GPU. If you hit out-of-memory (OOM) errors, you may use the documented fallback Qwen3-VL-4B. Using the 4B model is fully accepted and is not penalised, provided you state clearly which model you used and why.

3 Instruction - What You Must Build

Part A — task dataset and baseline (10 marks)

1. Load Qwen3-VL-8B-Instruct (4-bit) on a Colab T4 GPU using the starter notebook.
2. Create a task dataset (10 tasks)
3. Run the baseline: pass each task's image + question directly to the model (no tools, no loop) and record its answer.
4. Report on the baseline accuracy of the 10 tasks. This is your "no-agent" control.

Part B — ReAct agent (15 marks)

Implement the Thought - Action - Observation loop.

1. Implement **at least three tools** that are suited to the tasks you designed/selected, for example:
   - `crop_region(box)` - return a cropped sub-image for closer inspection
   - `read_text(box)` - ask the VLM to transcribe text in a region
   - `calculator(expression)` - evaluate an arithmetic expression safely
2. Use the model's native tool-calling interface (structured `tool_calls`) to decide the next action. Your framework code must parse the tool call, execute the tool, and feed the observation back into the conversation.
3. Loop until the agent reaches the answer or a maximum step limit is reached.

4. Report ReAct accuracy on the 10 tasks and compare against the Part A baseline.

Part C — Reflexion Agent (15 marks)

Add a Reflexion layer on top of your ReAct agent.

1. Evaluator: after an attempt, decide success/failure. Because the dataset ships with ground-truth answers, you may use a rule-based evaluator or an LLM-as-judge.
2. Self-Reflection: on failure, prompt the model to produce a *specific* verbal reflection (what went wrong, what to do differently).
3. Memory: store reflections and inject them into the context on the next attempt.
4. Retry: re-attempt each failed task up to `N` times (e.g. N = 3), carrying accumulated reflections forward.
5. Produce a **comparison table** showing accuracy: Baseline vs ReAct vs Reflexion.

Part D — Report (10 marks)

A concise report (max **4 pages**, PDF) containing:

1. Which model you used (8B or 4B fallback) and your Colab setup (quantisation, image resolution caps, any OOM handling).
2. Document the 10 tasks in your task dataset.
3. Tool design justification: what tools you chose and why.
4. At least 3 documented failure cases, with the reasoning trace, and an analysis of whether ReAct/Reflexion fixed them.
5. A **critical reflection on small-model agents**: where did the 8B/4B model struggle (parsing, hallucination, counting, arithmetic)? This connects to the open question: *"Can small models reason and act effectively?"*

4 Submission

Your team is to submit a single ZIP named `<studentID>_IT3103_ASSN2.zip` containing:

1. `agent_qwen.ipynb` - your completed notebook (Parts A-C), **with outputs saved** so the marker can see it ran.
2. `report.pdf` - your report (Part D).
3. `README.md` - one paragraph: which model, how to run, any known issues.
4. task images you created, in a `data/` folder.

Be prepared for a short demo, which will be conducted in Week 18.

4 Grading Rubric

Total Marks: 50 marks

Criteria	Inadequate (F)	Acceptable (D)	Satisfactory (C)	Good (B)	Excellent (A)
Task Dataset & Baseline	0 – 4.9 mark	5 – 5.9 marks	6 – 6.9 marks	7 – 7.9 marks	8 – 10 marks
	• No effort in searching for a suitable dataset for the assignment • The code for the baseline model has error	• Minimum effort in gathering a dataset for the assignment • No error with the code, baseline models work for all tasks	• Some effort in gathering a suitable dataset for the assignment • No error with the code, baseline models work for all tasks	• Good effort in gathering a good quality dataset for the assignment • No error with the code, baseline models work for all tasks	• Excellent effort in gathering a good quality and suitable dataset for the assignment • No error with the code, baseline models work for all tasks
ReAct Agent	0 – 7.4 marks	7.5 – 8.9 marks	9 – 10.4 marks	10.5 – 11.9 marks	12 – 15 marks
	• Agent doesn't work, or no working agent loop	• Limited/non-functional ReAct agent	• Working ReAct • No errors with the code • But no improvement over baseline	• Working ReAct • No errors with the code • With measurable improvement over baseline	• Working ReAct • No errors with the code • With measurable improvement over baseline
Reflexion Agent	0 – 7.4 marks	7.5 – 8.9 marks	9 – 10.4 marks	10.5 – 11.9 marks	12 – 15 marks
	• Agent doesn't work, or no reflection loop	• Limited/non-functional Reflexion agent	• Partly functional Reflexion	• Functional Reflexion with	• Functional Reflexion with

Criteria	Inadequate (F)	Acceptable (D)	Satisfactory (C)	Good (B)	Excellent (A)
				minor rough edges • measurable improvement over baseline	robust engineering; • measurable improvement over baseline
Report	0 – 4.9 mark	5 – 5.9 marks	6 – 6.9 marks	7 – 7.9 marks	8 – 10 marks
	• The writing is unclear or incoherent. • Lack of content and no discussion of results.	• The writing is reasonably clear. • minimum discussion of results.	• The writing is reasonably clear and coherent. • The content is general and contains some discussion of experimental results.	• The writing is clear and coherent. • The content is specific with good quality discussion of experimental results.	• The writing is clear and coherent. • The content is specific and insightful with high-quality discussion of experimental results.

BLANK